

Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales

Unidad de Aprendizaje:
Inteligencia Artificial
Grupo: 6CV2

Práctica: A*

Alumno:
Barros Martínez Luis Enrique
Bautista Ríos Alfredo
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

El código presentado implementa una solución para el algoritmo de búsqueda **A***, ampliamente utilizado en el ámbito de la inteligencia artificial y la informática para encontrar el camino más corto entre dos puntos en un espacio discretizado, como un mapa o una cuadrícula. Este algoritmo combina la búsqueda de menor costo con una heurística que aproxima la distancia restante al objetivo, logrando un equilibrio entre eficiencia y precisión. Su popularidad radica en su capacidad para encontrar soluciones óptimas en problemas de enrutamiento, planificación de movimientos en robots, y navegación en videojuegos, entre otros.

El programa utiliza la biblioteca SDL para la visualización y el manejo de eventos, lo que permite representar gráficamente el progreso del algoritmo en tiempo real. Además, organiza el flujo del programa en diversas funciones y clases que facilitan su comprensión y extensibilidad. Desde el manejo de colores y texturas hasta la lógica central del algoritmo, el código ofrece una visión integral de cómo estructurar una solución basada en A* y cómo presentar visualmente el proceso de búsqueda y su resultado.

Desarrollo

Estructura del Programa

El programa consta de tres componentes principales:

1. Manejo de la visualización con SDL:

La clase `vsr::Screen` se encarga de manejar la ventana y renderizar los elementos visuales, como líneas de la cuadrícula, texturas, y símbolos de inicio, objetivo y bloqueos. Esta parte del programa proporciona una interfaz gráfica que ilustra la cuadrícula y permite al usuario interactuar con el sistema mediante clics del mouse.

2. Algoritmo A*:

La lógica del algoritmo está implementada en la clase `Pathfinder`, que gestiona una cuadrícula donde cada celda puede ser transitable o no. Esta clase encapsula métodos clave como:

- `Get_lower_open()`: Encuentra el nodo con el menor costo heurístico en la lista abierta.

- `Check_neighbours()`: Explora los vecinos de un nodo actual, actualizando los costos acumulados y la lista abierta.
- `Step()`: Ejecuta un paso del algoritmo, moviendo nodos de la lista abierta a la cerrada y calculando caminos. Estas funciones cooperan para garantizar que el algoritmo avance de manera óptima desde el punto inicial hasta el objetivo.

3. Renderizado de Resultados y Proceso de Iteración:

El bucle principal del programa combina los datos del algoritmo con funciones de visualización para mostrar el estado actual de la búsqueda. Se representan elementos como:

- Las celdas exploradas, mediante flechas que indican la dirección del movimiento.
- El camino encontrado, coloreado en verde.
- Los nodos de inicio y destino, representados por íconos específicos.

Detalles del Algoritmo

El algoritmo implementado se basa en celdas que almacenan información como:

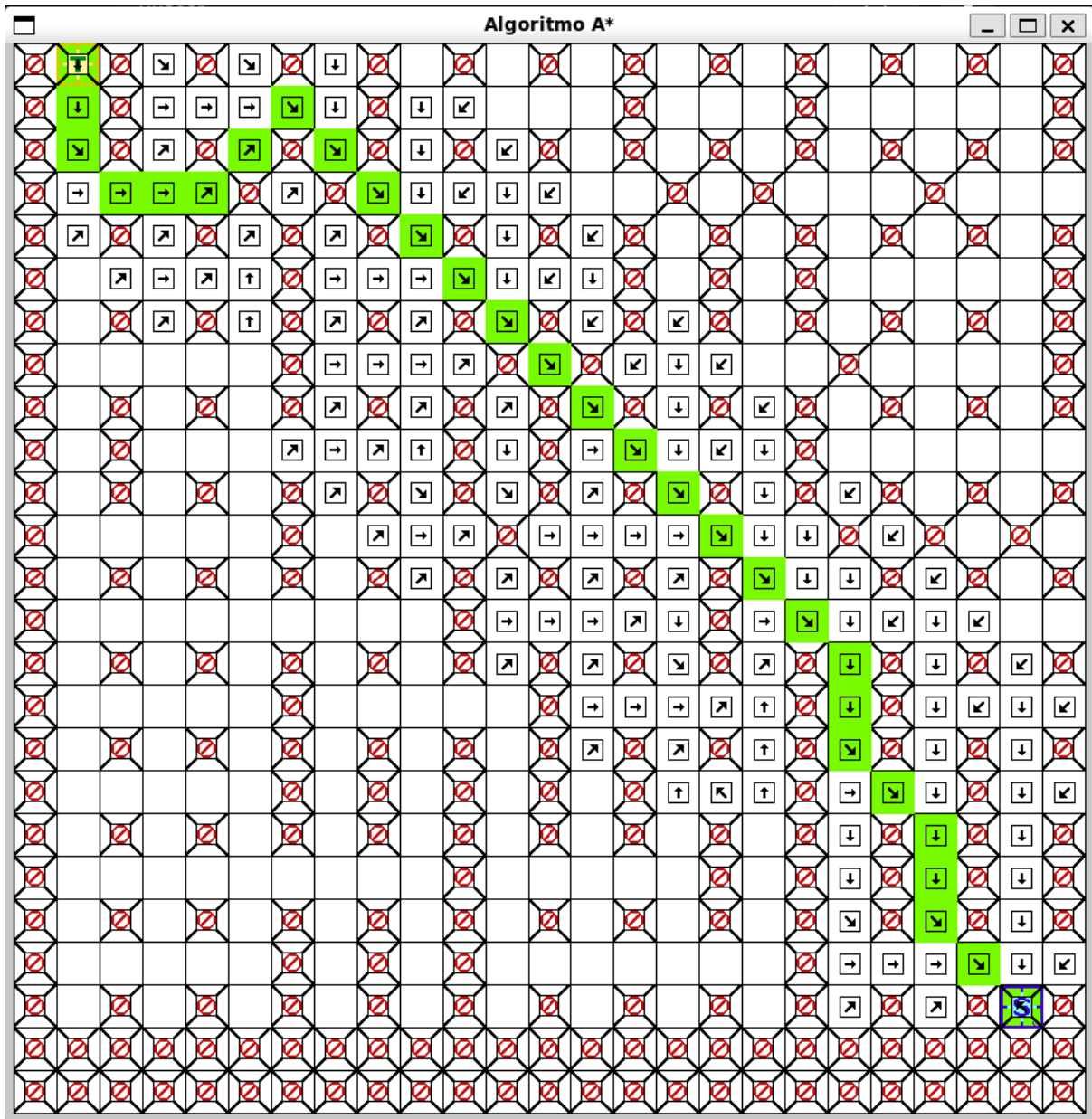
- Su posición en la cuadrícula.
- Un puntero al nodo padre para reconstruir el camino.
- Los costos acumulados (**g**), heurísticos (**h**), y totales (**f = g + h**). Cada celda es instanciada como un objeto de la clase `A_star_cell`, que proporciona métodos para actualizar su estado y determinar el índice de textura basado en la dirección del movimiento.

Gestión de Mapas y Texturas

El archivo `map.txt` contiene la representación inicial del mapa. Este se carga en `Pathfinder`, donde se genera una máscara de accesibilidad (`grid_mask`) y una representación interna para calcular caminos. Además, las texturas para flechas y otros elementos gráficos se organizan en listas, permitiendo una representación clara de los pasos del algoritmo y su resultado.

Con esta base, el programa se adapta fácilmente a diferentes tamaños de mapas, heurísticas y configuraciones visuales, haciéndolo útil para explorar y experimentar con variantes del algoritmo A*.

Ejecución



Conclusión

El código desarrollado ofrece una implementación robusta y visualmente clara del algoritmo A*, mostrando cómo estructurar un programa eficiente y modular para resolver problemas de enrutamiento en una cuadrícula. Al integrar bibliotecas como SDL para la visualización y dividir las responsabilidades en clases y funciones específicas, el programa no solo logra cumplir su objetivo funcional, sino que también sirve como un ejemplo didáctico para aprender sobre algoritmos de búsqueda y su representación gráfica.

Además, su diseño modular permite la extensibilidad, pudiendo incorporar diferentes heurísticas, mapas o métodos de visualización sin alterar significativamente la estructura base. Esto lo convierte en una herramienta versátil tanto para el aprendizaje como para aplicaciones prácticas en áreas como videojuegos, robótica y planificación de rutas. En conjunto, el proyecto destaca por su claridad, funcionalidad y potencial para futuras expansiones o adaptaciones.

Código

A_star.c++

```
#include "Pathfinder.hpp"
#include "../SDL_Visor/SDL_Visor.hpp"

const int WIDTH = 800;
const int HEIGHT = 800;

void eventos(SDL_Event& eventos);
std::map<String, vsr::Color*> Load_colors();
Vector<vsr::Icon*> Load_arrow_Icons(vsr::Screen* window);
void create_map_texture(String texture_name, uint16_t
cell_widht, Vector<Vector<bool>> map_grid, vsr::Screen* window, vsr::Icon
blocked_icon, vsr::Color &grid_color);
double heuristic(Cord target, Cord current);
Cord start, target;
uint8_t status = 0;
float escala = float(WIDTH/8);
Vector<Vector<bool>> mask;
int main(int argc, char const *argv[])
{
    //Create window
    vsr::Screen window("Algoritmo A*", WIDTH, HEIGHT, SDL_RENDERER_ACCELERATED);
    window.Set_events_handler(eventos);

    window.Init_TTF("sans", "../SDL_Visor/fonts/NotoSans/NotoSans-Thin.ttf", 10);
    window.Init_IMG();
    //Load colors
    auto colors = Load_colors();
    //Load Icons
    auto arrow_icons = Load_arrow_Icons(&window);
    vsr::Icon block_icon("../Iconos/BLOCK.png", window.Get_renderer());
    vsr::Icon start_icon("../Iconos/START.png", window.Get_renderer());
    vsr::Icon target_icon("../Iconos/TARGET.png", window.Get_renderer());
    //Load map_mask
    auto grid = Pathfinder("map.txt", heuristic);
    mask = grid.Get_grid_mask();
    escala = escala / mask.size();
    //Create background texture
```

```

uint16_t cell_size = 32;
create_map_texture("background", cell_size, grid.Get_grid_mask(),
&window, block_icon,*colors["Black"]);
Vector<Vector<uint8_t>> textures;
bool path_found = false;
int x,y;
uint16_t cell_width = WIDTH/mask.size();
uint16_t cell_height = HEIGHT/ mask[0].size();
SDL_Rect area;
while (window.Handle_events())
{
    window.Clean_screen(*colors["White"]);
    window.Draw_saved_texture("background");
    switch (status)
    {
        case 2:
            grid.Set_start(start);
            grid.Set_target(target);
            status++;
            break;
        case 3:
            if (!path_found)
            {
                path_found = grid.Step();
                if (path_found)
                {
                    status++;
                }
            }
            area =
{start.x*cell_width,start.y*cell_height,cell_width,cell_height};
            window.Draw_texture(start_icon.Get_texture(),&area);
            area =
{target.x*cell_width,target.y*cell_height,cell_width,cell_height};
            window.Draw_texture(target_icon.Get_texture(),&area);
            textures = grid.Get_textures();
            x = 0;
            for (auto &&column : textures)
            {
                y = 0;
                for (auto &&cell : column)
                {
                    if (cell != 255)
                    {
                        area =
{x*cell_width,y*cell_height,cell_width,cell_height};
window.Draw_texture(arrow_icons[cell]->Get_texture(),&area);
                    }

                    y++;
                }
            }
        }
    }
}

```

```

        }
        x++;
    }

    break;
default:

    Vector<Cord> path = grid.Get_path();

    for (auto &&path_cell : path){

        area = {path_cell.x
*cell_width,path_cell.y*cell_height,cell_width,cell_height};
        window.Draw_filled_rectangle(path_cell.x
*cell_width,path_cell.y*cell_height,cell_width,cell_height,*colors["Green"]);
    }
    area =
{start.x*cell_width,start.y*cell_height,cell_width,cell_height};
    window.Draw_texture(start_icon.Get_texture(),&area);
    area =
{target.x*cell_width,target.y*cell_height,cell_width,cell_height};
    window.Draw_texture(target_icon.Get_texture(),&area);

    x = 0;
    for (auto &&column : textures)
    {
        y = 0;
        for (auto &&cell : column)
        {
            if (cell != 255)
            {
                SDL_Rect area =
{x*cell_width,y*cell_height,cell_width,cell_height};
window.Draw_texture(arrow_icons[cell]->Get_texture(),&area);
            }

            y++;
        }
        x++;
    }
    break;
}

window.Present_renderer();

SDL_Delay(300);
}

return 0;

```

```

}

void eventos(SDL_Event& evento) {
switch (evento.type)
{
case SDL_MOUSEBUTTONDOWN:
    switch (evento.button.button)
    {
        case SDL_BUTTON_LEFT:
            std::cout << floor(evento.button.x/(escala*8)) << " " <<
floor(evento.button.y/(escala*8)) << std::endl;
            switch (status)
            {
                case 0:
                    start.x = evento.button.x/(escala*8);
                    start.y = evento.button.y/(escala*8);
                    if (mask[start.x][start.y])
                    {
                        status++;
                    }

                    break;
                case 1:
                    target.x = evento.button.x/(escala*8);
                    target.y = evento.button.y/(escala*8);
                    if (mask[target.x][target.y])
                    {
                        status++;
                    }
                    break;
                default:
                    break;
            }
            break;

        default:
            break;
    }
    break;

default:
    break;
}
}

std::map<String,vsr::Color*> Load_colors() {
    std::map<String, vsr::Color*> colors = {
        {"White", new vsr::Color{255, 255, 255, 255}},
        {"Black", new vsr::Color(0, 0, 0, 255)},
        {"Green", new vsr::Color(124, 252, 0, 255)},
        {"Orange",new vsr::Color(255, 128, 0, 255)},
        {"Red", new vsr::Color(237, 64, 64, 255)}
    }
}

```

```

};
return colors;
}

Vector<vsr::Icon *> Load_arrow_Icons(vsr::Screen* window)
{
    Vector<vsr::Icon *> icons;
    icons.push_back(new
vsr::Icon("../Iconos/ULA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/UA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/URA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/RA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/DRA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/DA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/DLA.png",window->Get_renderer()));
    icons.push_back(new
vsr::Icon("../Iconos/LA.png",window->Get_renderer()));
    return icons;
}

void create_map_texture(String texture_name, uint16_t cell_size,
Vector<Vector<bool>> map_grid, vsr::Screen *window, vsr::Icon blocked_icon,
vsr::Color &grid_color){
    uint16_t width = cell_size * map_grid.size();
    uint16_t height = cell_size * map_grid[0].size();
    window->Create_texture(texture_name, width+1, height+1);
    window->Clean_screen();
    // Dibujar líneas verticales
    for (uint16_t i = 0; i <= map_grid.size(); i++){
        window->Draw_line_pos(i * cell_size, 0, i * cell_size, height,
grid_color);
    }
    // Dibujar líneas horizontales
    for (uint16_t i = 0; i <= map_grid[0].size(); i++){
        window->Draw_line_pos(0, i * cell_size, width, i * cell_size,
grid_color);
    }
    for (uint16_t u = 0; u < map_grid[0].size(); u++)
    {
        for (uint16_t i = 0; i < map_grid.size(); i++)
        {
            if(!map_grid[u][i]){
                SDL_Rect block_area =
{u*cell_size,i*cell_size,cell_size,cell_size};
                window->Draw_texture(blocked_icon.Get_texture(),&block_area);
            }
        }
    }
}

```

```

    }

    window->End_texture();
}

double heuristic(Cord target, Cord current){
    return std::max(std::abs(target.x - current.x), std::abs(target.y -
current.y));
}

```

Pathfinder

```

#include "Pathfinder.hpp"

Cord Pathfinder::Get_lower_open(){
    if (open.empty()) {
        return { UINT16_MAX, UINT16_MAX }; // Invalid coordinate
    }

    // Initialize with the first element
    Cord lowest_cord = open.front();
    uint lowest_weight = grid[lowest_cord.x][lowest_cord.y]->Get_weight();

    // Find the node with the lowest weight
    for (Cord& c : open) {
        uint current_weight = grid[c.x][c.y]->Get_weight();
        if (current_weight < lowest_weight) {
            lowest_weight = current_weight;
            lowest_cord = c;
        }
    }
    return lowest_cord;
}

//void Pathfinder::Check_neighbours(Cord current){}

void Pathfinder::Check_neighbours(Cord current) {
    static const int dx[] = { -1, -1, -1, 0, 0, 1, 1, 1 };
    static const int dy[] = { -1, 0, 1, -1, 1, -1, 0, 1 };

    for (int dir = 0; dir < 8; ++dir) {
        int nx = current.x + dx[dir];
        int ny = current.y + dy[dir];

        // Boundary checks
        if (nx < 0 || ny < 0 || nx >= grid_mask.size() || ny >=
grid_mask[0].size()) {
            continue;
        }

        Cord neighbour = { static_cast<uint16_t>(nx),
static_cast<uint16_t>(ny) };

        // Skip if not passable or already closed
        if (!grid_mask[nx][ny] || std::find(closed.begin(), closed.end(),
neighbour) != closed.end()) {
            continue;
        }
    }
}

```

```

        uint tentative_g = grid[current.x][current.y]->Get_accumulated() + 1;
// Assuming uniform cost

        // If neighbor is not in open list, add it
        if (std::find(open.begin(), open.end(), neighbour) == open.end()) {
            grid[nx][ny] = new A_star_cell(current, neighbour, target,
tentative_g, heuristic_function);
            grid[nx][ny]->Set_parent(current);
            open.push_back(neighbour);
        } else {
            // Update if a better path is found
            if (tentative_g < grid[nx][ny]->Get_accumulated()) {
                grid[nx][ny]->Update(tentative_g, tentative_g +
grid[nx][ny]->Get_heuristic(), current);
            }
        }
    }
}

uint16_t Pathfinder::A_star_cell::last_id = 0;
Pathfinder::Pathfinder(String map_file_path)
{
    start_counter = false;
    path_found = false;
    std::ifstream infile(map_file_path);
    if (!infile.is_open()) {
        std::cerr << "Error: Can't open file: " << map_file_path <<
std::endl;
        return;
    }

    String line;
    // Obtener dimensiones
    if (std::getline(infile, line)) {
        std::istringstream iss(line);
        int rows, cols;
        if (!(iss >> rows >> cols)) {
            std::cerr << "Error: Wrong format for the header in file: " <<
map_file_path << std::endl;
            infile.close();
            return;
        }

        grid_mask.resize(cols, Vector<bool>(rows, false));
        grid.resize(cols, Vector<A_star_cell*>(rows, nullptr));

        // Load Map
        int current_row = 0;
        while (std::getline(infile, line) && current_row < rows) {
            if (line.length() < static_cast<size_t>(cols)) {
                std::cerr << "Warning: line " << current_row + 2 << " is
incomplete. Filling with '#' " << std::endl;
            }
            for (int col = 0; col < cols; ++col) {
                if (col < static_cast<int>(line.length())) {
                    char ch = line[col];
                    if (ch != '#') {
                        grid_mask[col][current_row] = true;
                    }
                } else {

```

```

        grid_mask[col][current_row] = false;
    }
}
current_row++;
}
// Validate size
if (current_row < rows) {
    std::cerr << "Warning: Map dimensions are wrong. Filling with
'#'." << std::endl;
}
} else {
    std::cerr << "Error: void file: " << map_file_path << std::endl;
}
infile.close();
start_counter = 0;
}

Pathfinder::Pathfinder(String map_file_path, double (*heuristic) (Cord target,
Cord current)):Pathfinder(map_file_path) {
    heuristic_function = heuristic;
}

Vector<Vector<bool>> Pathfinder::Get_grid_mask() {
    return grid_mask;
}

void Pathfinder::Set_start(Cord start) {
    this->start = start;
    if (start_counter < 2) {
        start_counter++;
    }
}

void Pathfinder::Set_target(Cord end) {
    this->target = end;
    if (start_counter < 2) {
        start_counter++;
    }
}

bool Pathfinder::Step() {
//std::cout << "DF 1" <<std::endl;
    if (start_counter < 2) {
        std::cerr << "Error: Can't start, missing start or target" <<
std::endl;
        return false;
    }
//std::cout << "DF 2" <<std::endl;
    // Initialize the start node if the open list is empty
    if (open.empty()) {
//std::cout << "DF 2.0" <<std::endl;
        grid[start.x][start.y] = new A_star_cell(start, start, target, 0,
heuristic_function);
//std::cout << "DF 2.1" <<std::endl;
        open.push_back(start);
    }
//std::cout << "DF 3" <<std::endl;
    // No path exists if the open list is empty after initialization
    if (open.empty()) {
        std::cerr << "Error: No path found." << std::endl;
        return false;
    }
}

```

```

    }
    //std::cout << "DF 4" <<std::endl;
    // Get the node with the lowest weight (f = g + h) from the open list
    Cord current = Get_lower_open();
    //std::cout << "DF 5" <<std::endl;
    // Check if the target is reached
    // std::cout << current.x << " " << current.y << "target " << target.x <<
    " " << target.y<< std::endl;

    if (current.x == target.x && current.y == target.y) {
        //std::cout << "Path found!" << std::endl;
        path_found = true;
        // Optional: Reconstruct path here by backtracking using parent nodes
        return true;
    }
    //std::cout << "DF 6" <<std::endl;
    // Remove current node from the open list
    //std::cout << "Removed from open " << current.x << " " << current.y <<
    std::endl;
    open.erase(std::remove(open.begin(), open.end(), current), open.end());
    //std::cout << "DF 7" <<std::endl;
    // Add current node to the closed list
    closed.push_back(current);
    //std::cout << "DF 8" <<std::endl;
    // Explore neighbors
    static const int dx[] = { -1, -1, -1, 0, 0, 1, 1, 1 };
    static const int dy[] = { -1, 0, 1, -1, 1, -1, 0, 1 };
    //std::cout << "DF 9" <<std::endl;
    for (int dir = 0; dir < 8; ++dir) {
        int nx = current.x + dx[dir];
        int ny = current.y + dy[dir];
    //std::cout << "DF 10" <<std::endl;
        // Check boundaries
        if (nx < 0 || ny < 0 || nx >= grid_mask.size() || ny >=
grid_mask[0].size()) {
            continue;
        }
    //std::cout << "DF 11" <<std::endl;
        Cord neighbour = { static_cast<uint16_t>(nx),
static_cast<uint16_t>(ny) };

        // Skip if the cell is not passable or already in the closed list
        if (!grid_mask[nx][ny] || std::find(closed.begin(), closed.end(),
neighbour) != closed.end()) {
            continue;
        }
    //std::cout << "DF 12" <<std::endl;
        double tentative_g = grid[current.x][current.y]->Get_accumulated() +
Distance(current,neighbour);
    //std::cout << "DF 13" <<std::endl;
        // If the neighbor is not in the open list, create a new A_star_cell
        if (std::find(open.begin(), open.end(), neighbour) == open.end()) {
            grid[nx][ny] = new A_star_cell(current, neighbour, target,
tentative_g, heuristic_function);
            open.push_back(neighbour);
        } else {
            // If the neighbor is already in the open list, check if the new
path is better
    //std::cout << "Tentative" << tentative_g << " Current: " <<
grid[nx][ny]->Get_accumulated() << std::endl;
            if (tentative_g <= grid[nx][ny]->Get_accumulated()) {

```

```

        grid[nx][ny]->Update(tentative_g,tentative_g +
grid[nx][ny]->Get_heuristic(),current);
    }
}
}
return false;
}

Vector<Vector<uint8_t>> Pathfinder::Get_textures() {
    // Initialize a 2D vector of uint8_t with the same size as the grid
    Vector<Vector<uint8_t>> textures(grid.size(),
Vector<uint8_t>(grid[0].size(), 0));

    // Iterate through the grid to populate the textures
    for (size_t x = 0; x < grid.size(); ++x) {
        for (size_t y = 0; y < grid[x].size(); ++y) {
            if (grid[x][y] != nullptr) {
                // Assign the texture value of the corresponding A_star_cell
                textures[x][y] = grid[x][y]->Get_texture();
            } else {
                // Assign a default value (e.g., 0 for uninitialized cells)
                textures[x][y] = 255;
            }
        }
    }

    return textures;
}

Vector<Cord> Pathfinder::Get_path()
{
    auto current = target;
    Vector<Cord>path;
    if (path_found)
    {
        while (current != start)
        {
            path.push_back(current);
            current = grid[current.x][current.y]->Get_parent();
        }
        path.push_back(current);
    }
    return path;
}

Pathfinder::~Pathfinder() {
}

uint8_t Pathfinder::A_star_cell::GetTextureIndex(int dx, int dy)
{
    if (dx == -1 && dy == -1) return 0; // UP LEFT ARROW
    if (dx == 0 && dy == -1) return 1; // UP ARROW
    if (dx == 1 && dy == -1) return 2; // UP RIGHT ARROW
    if (dx == 1 && dy == 0) return 3; // RIGHT ARROW
    if (dx == 1 && dy == 1) return 4; // DOWN RIGHT ARROW
    if (dx == 0 && dy == 1) return 5; // DOWN ARROW
    if (dx == -1 && dy == 1) return 6; // DOWN LEFT ARROW
    if (dx == -1 && dy == 0) return 7; // LEFT ARROW
    return 0; // Default case;
}

```

```

}

Pathfinder::A_star_cell::A_star_cell(Cord parent, Cord position, Cord target,
uint accumulated, double (*heuristic)(Cord target, Cord current))
{
    if (heuristic == nullptr)
    {
        std::cerr << "Error: heuristic is nullptr" << std::endl;
    }

    //std::cout << "DF A1" <<std::endl;
    this->heuristic = heuristic(target, position);
    //std::cout << "DF A1.1" <<std::endl;
    this->accumulated = accumulated;
    this->parent = parent;
    this->position = position;
    //std::cout << "DF A2" <<std::endl;
    weight = accumulated + this->heuristic;

    ID = last_id;
    last_id++;

    // Calculate direction from current node to parent node
    int dx = parent.x - position.x;
    int dy = parent.y - position.y;
    //std::cout << "DF A3" <<std::endl;
    // Map direction to texture index
    texture = GetTextureIndex(dx, dy);
    //std::cout << "DF A4" <<std::endl;
}

void Pathfinder::A_star_cell::Set_parent(Cord new_parent){
    parent = new_parent;
}

void Pathfinder::A_star_cell::Update(uint new_accumulated, uint new_weigth,
Cord new_parent){
    accumulated = new_accumulated;
    weight = new_weigth;
    parent = new_parent;
    int dx = parent.x - position.x;
    int dy = parent.y - position.y;
    texture = GetTextureIndex(dx, dy);
}

double Pathfinder::A_star_cell::Get_accumulated()
{
    return accumulated;
}

double Pathfinder::A_star_cell::Get_heuristic()
{
    return heuristic;
}

double Pathfinder::A_star_cell::Get_weight()
{
    return weight;
}

Cord Pathfinder::A_star_cell::Get_parent()

```

```
{
    return Cord(parent);
}

uint8_t Pathfinder::A_star_cell::Get_texture()
{
    return texture;
}

Pathfinder::A_star_cell::~A_star_cell()
{
}

double Distance(const Cord& point1, const Cord& point2) {
    // Use the distance formula: sqrt((x2 - x1)^2 + (y2 - y1)^2)
    return std::sqrt(std::pow(point2.x - point1.x, 2) + std::pow(point2.y -
point1.y, 2));
}
```